StealthChat: The Complete Team Study & Viva Guide

Master Guide for Presentation, Technical Terminology & Oral Examination (Viva)

> 🎯 **Purpose of this Guide for Teammates:**
> Read this document before our teacher questions us! It explains **every piece of technical jargon from the absolute basics**, details the architecture of what we built, explains how our research was conducted, breaks down the **core novelty (our image steganography encryption)**, and gives you exact word-for-word answers to every trick question the teacher might throw at you.

1. The 30-Second Elevator Pitch (Memorize This!)

> **If the teacher points at you and says: "Tell me what your project is about in 30 seconds":**
> *"Professor, our project is called **StealthChat**. It explores real-time multi-client communication across the networking stack. We started by building a low-level Java TCP socket server to understand raw byte streams and thread concurrency. We then evolved it into an enterprise full-stack web chat system utilizing **WebSockets**, **Pub/Sub message fan-out**, **JWT channel authorization**, and **IndexedDB offline caching**. Most importantly, our **core novelty** is an in-browser **Steganographic Carrier Encryption** method: instead of sending obvious ciphertext that triggers firewall alerts, we hide secret messages inside procedural images, completely concealing the communication from Deep Packet Inspection."*

2. Jargon Buster: Every Networking Term Explained Simply

Here are the fundamental networking concepts used in this project, explained without confusing textbook clutter:

🔌 Socket

The software endpoint that lets programs talk over a network. Think of it as a virtual plug socket connecting two computers over the internet.

🚪 Port (e.g. 8189 vs 8080)

A logical door number on your computer. IP address gets data to the machine; the Port number tells the OS which specific application should receive it.

🤝 TCP vs UDP

TCP is connection-oriented and reliable (3-way handshake, in-order packets, auto-retransmits). **UDP** is fast but fire-and-forget (packets can drop or get scrambled). Chat apps *must* use TCP.

🧵 Concurrency & Multithreading

If a server has only 1 thread, it can only talk to 1 user at a time (all others freeze). We spawn worker threads (`ClientHandler`) so hundreds of users can talk at once.

⚡ WebSockets (RFC 6455)

Unlike normal HTTP where the browser asks and waits, WebSockets establish a **permanent, 2-way highway**. The server can push messages instantly to the browser with only 2–10 bytes of frame overhead.

📣 Pub/Sub (Publish-Subscribe)

A messaging pattern where senders "publish" messages to a topic (like `/topic/101`) and all subscribed users receive it automatically without knowing who the sender is directly.

🎟️ JWT (JSON Web Token)

A cryptographically signed identity badge. When a user logs in, they get a JWT token. They show this token on every WebSocket connection to prove who they are without querying passwords.

🛡️ Channel Interceptor

A security gatekeeper inside the WebSocket pipeline. When a user tries to subscribe to a chat room, the interceptor checks the database: if you're not in the group, it drops your packet.

💓 Heartbeat & Cache TTL

Clients send a ping every 15s. If the server doesn't get a ping within 30s (Time-To-Live), it knows the user lost power or signal and marks them `OFFLINE`.

🔁 Idempotency & Deduplication

On mobile connections, duplicate packets happen when ACKs drop. The client assigns a UUID (`clientMsgId`); the server remembers it for 5 min and ignores duplicate copies.

📑 BroadcastChannel API

A browser feature allowing multiple tabs to talk directly to each other without pinging the server. If Tab 1 reads a message, Tab 2 updates its blue ticks instantly.

📦 IndexedDB & Service Worker

IndexedDB is an in-browser database caching chat history locally. **Service Worker** intercepts offline requests and background-syncs queued messages once Wi-Fi returns.

3. The Core Novelty: Steganographic Carrier Image Encryption

> ⭐ **THIS IS THE MOST IMPORTANT SECTION OF THE PROJECT!**
> The entire grade depends on explaining why our project is **novel and innovative** compared to other chat apps.

The Critical Flaw of Apps like WhatsApp, Signal, and Telegram:

- WhatsApp and Signal use state-of-the-art End-to-End Encryption (E2EE) such as AES-256-GCM.
- **However, the ciphertext itself is loudly visible on the network wire.** When encrypted text traverses an ISP or firewall, it has high *Shannon entropy* (it looks like random high-entropy noise).
- **Deep Packet Inspection (DPI)** appliances and network administrators immediately detect that an encrypted, secretive conversation is happening. In authoritarian regions, college networks, or corporate firewalls, encrypted streams are routinely blocked, flagged, or tracked.

Our Novel Innovation ("StealthChat" Protocol):

Instead of sending obvious ciphertext that triggers alarms, our application invents a **Zero-Knowledge Steganographic Carrier Channel**:

1. **Procedural Canvas Generation:** When a user turns on "Stealth Mode", the browser generates a unique, random color gradient on an HTML5 `<canvas>` element. (Zero hardcoded images; every message has a unique visual fingerprint).
2. **Binary Stream Conversion:** The secret message is converted into UTF-8 binary bits, terminated with a 16-bit null terminator (`0000000000000000`).
3. **Bitwise Least Significant Bit (LSB) Encoding:** We iterate over the Red, Green, and Blue bytes of the pixel data (skipping Alpha to keep opacity 100%), and replace the lowest bit:

   ```
   // The Core Bitwise Algorithm in index.html: data[i] = (data[i] & ~1) | parseInt(secretBit);
   ```

4. **Imperceptible Visual Variance (≤ 0.39%):** Changing only the lowest bit changes a color channel by at most **1 out of 255**. The human visual cortex cannot perceive this tiny shift, and computer heuristic filters see standard pixel noise.
5. **Network Camouflage:** The WebSocket transmits a legitimate `image/png` Base64 data payload. To network sniffers (Wireshark), firewalls, and database logs, **it looks 100% like someone simply shared a random piece of digital artwork**!
6. **Client-Side GPU Extraction:** The recipient receives the image and decrypts it purely inside their local canvas buffer. The server never sees the plaintext or the keys.

Competitive Analysis Table (Show this to your teacher!):

Architectural Metric	WhatsApp / Signal	Telegram / Slack	StealthChat (Our Innovation)
Payload Visibility to DPI	Obvious Ciphertext stream	Cleartext / Ciphertext	**100% Camouflaged as a Benign PNG Image**
Deniable Communication	No (Cryptographic trail proves secret talk)	No	**Yes (User can claim they shared digital art)**
Network Signature	Flagged as encrypted traffic	Standard HTTPS/REST	**Indistinguishable from normal image sharing**
Client Overhead	Heavy compiled native libraries	Standard Web / App SDK	**Zero-Plugin Native HTML5 Canvas + WebSockets**
Delivery Receipts	Sent / Delivered / Read Ticks	Sent / Read	**Sent (✓), Delivered (✓✓), Read (✓✓ Blue)**
Offline Resilience	Native SQLite Database	Server-only Cloud Sync	**Native IndexedDB + Service Worker Sync**

4. What Did We Actually Build? (The Architecture Roadmap)

Our project consists of two distinct engineering milestones:

Phase 1: The Raw Java Socket Prototype (Computer Networks Foundation)

- **`ChatServer.java`:** Binds a raw TCP socket on port `8189`. Runs an infinite loop accepting connections via `serverSocket.accept()`. Dispatches each client to a `ClientHandler` thread. Maintains a thread-safe `synchronized(clientWriters)` set to broadcast text to all connected users.
- **`ChatClient.java`:** Terminal console client. Uses two threads: main thread reads keyboard input, listener thread reads incoming socket text.
- **`ChatClientGUI.java`:** Desktop graphical client built with Java Swing. Demonstrates how the Event Dispatch Thread (EDT) updates the window while a background worker thread listens to the socket to prevent UI freezing.

Phase 2: The Enterprise StealthChat Web Application (Modern Real-Time Architecture)

Built with **Spring Boot 3.2.3, Java 17, and HTML5/Vanilla JS**:

- **WebSockets & STOMP Broker (`WebSocketConfig.java`):** Enables the `/ws-chat` endpoint with SockJS and configures the in-memory message broker on `/topic`.
- **Packet-Level Security Interceptor (`WebSocketSecurityChannelInterceptor.java`):** Validates JWT on `CONNECT` and verifies user membership in `GroupMembershipRepository` on `SUBSCRIBE`.
- **Idempotency & Deduplication Engine (`IdempotencyService.java`):** Sliding 5-minute cache that drops duplicate packets carrying the same client-generated UUID (`clientMsgId`).
- **Presence & Heartbeat Engine (`PresenceService.java`):** Maintains user online status with a 30-second TTL cache, supported by a `@Scheduled` background eviction sweep every 5 seconds.
- **Delivery & Read State Machine (`ChatController.java`):** Handles `/app/chat.sendMessage`, `/app/chat.ackDelivery`, and `/app/chat.ackRead` to power the dynamic WhatsApp-style tick indicators.
- **Offline Resilient Client (`index.html + sw.js`):** Stores room messages in **IndexedDB** (`StealthChatDB`) and uses a **Service Worker** with the Background Sync API to queue and auto-flush messages when disconnected.

5. The Live Demonstration Script (What to Click & What to Say)

> **Follow this exact 4-step script during your demonstration:**

1. **Step 1: Open Two Windows Side-by-Side (Real-Time Communication)**
   Open `http://localhost:8080` in two side-by-side tabs. Log into Tab 1 as `admin` (password: `admin123`). In Tab 2, register as `alice`. Both join `GLOBAL_1`.

What to say: "Here we have two concurrent clients communicating over persistent WebSockets using the STOMP publish-subscribe protocol."

2. **Step 2: Show the Delivery Ticks Progression**
   Send a message from Admin. Show how the tick next to the message changes from single gray ✓ (Sent) to double gray ✓✓ (Delivered to Alice) to double blue ✓✓ (Read when Alice opens the chat).
   What to say: "This demonstrates our bidirectional ACK state machine. The recipient client automatically fires delivery and read receipts back across the broker."
3. **Step 3: Trigger the Core Novelty (Stealth Mode)**
   In Admin's window, click the eye/shield icon. Type: `"Top Secret Payload"`. Press Enter.
   Admin's window sends a colorful canvas image. Alice receives the image with a button **"🔓 Decrypt Carrier Image"**. Click it: the secret text appears!
   What to say: "This is our core innovation. The secret text was embedded into the Least Significant Bits of the image pixels. To any network monitor or firewall, it looked like normal image sharing."
4. **Step 4: Show the Admin Audit Interception Log**
   In Admin's window, click **"⚙️ Admin"**. Point to the intercepted logs: it shows `[BASE64 CARRIER IMAGE - 14820 bytes]`.
   What to say: "Notice that even with database access, the network observer only captures the carrier image. The secret plaintext was never exposed on the wire."

6. Teacher Viva / Exam Defense Handbook

Here are the exact questions teachers love to ask, along with the high-scoring answers:

❓ Q1: "Why did you use TCP instead of UDP for the chat application?"

> **Answer:** "Instant messaging requires absolute reliability and sequential ordering. TCP provides a 3-way handshake, packet sequence numbers, error checksums, and automatic retransmissions. If we used UDP, packets could be lost or arrive scrambled—altering conversation meaning (e.g., dropping critical numbers or words)."

❓ Q2: "How is WebSockets different from traditional HTTP polling?"

> **Answer:** "In HTTP polling, the client repeatedly sends requests every few seconds. Each request incurs 500B to 2KB of redundant header overhead and high server CPU load. WebSockets performs an initial HTTP 101 Upgrade handshake, then establishes a persistent, full-duplex TCP stream with a framing overhead of only **2 to 10 bytes**."

❓ Q3: "What happens if a user's phone battery dies? How does the server know they are offline?"

> **Answer:** "An abrupt power loss or signal drop doesn't send a clean TCP FIN packet. To solve this, we implemented a **Heartbeat & TTL Cache** in `PresenceService.java`. Clients ping `/app/chat.heartbeat` every 15s. If no ping is received within **30 seconds (TTL)**, a background `@Scheduled` sweeper automatically evicts the user and broadcasts `OFFLINE` to the channel."

❓ Q4: "What prevents duplicate messages when a mobile user clicks Send multiple times on a bad connection?"

> **Answer:** "We implemented **Client-Side Idempotency**. The browser generates a unique UUID (`clientMsgId`) before transmitting. The backend's `IdempotencyService.java` maintains a 5-minute sliding TTL cache. If a duplicate packet arrives with the same ID, the server drops the duplicate database write and merely returns the existing ACK."

❓ Q5: "How do you stop unauthorized users from snooping on private groups?"

> **Answer:** "We built a `WebSocketSecurityChannelInterceptor.java`. When a client sends a STOMP `CONNECT` frame, we validate their signed HMAC-SHA256 JWT. When they send a `SUBSCRIBE` frame to `/topic/{roomId}`, the interceptor checks `GroupMembershipRepository`. If they are not an authorized member, the subscription packet is rejected."

❓ Q6: "What is your project's novelty and how does the image encryption work?"

> **Answer:** "Our novelty is **Steganographic Carrier Encryption** to defeat Deep Packet Inspection (DPI). While WhatsApp and Signal send obvious ciphertext that firewalls flag, we generate a dynamic HTML5 Canvas artwork and encode secret bits into the Least Significant Bit (LSB) of the RGB channels using `(pixel & ~1) | bit`. The maximum color variance is under 0.39%—invisible to the human eye. To the network, it looks like innocent PNG image sharing."

❓ Q7: "What happens when the client is completely offline?"

> **Answer:** "We built an **Offline-First Architecture** using **IndexedDB** (`StealthChatDB`) and a **Service Worker** (`sw.js`). Outgoing messages are queued in an IndexedDB outbox with a 🕑 pending icon. When the network reconnects, the Service Worker Background Sync triggers and automatically flushes the queue over the WebSocket."

❓ Q8: "Why did you use IndexedDB instead of localStorage?"

> **Answer:** "`localStorage` is synchronous, blocks the browser UI thread, and is capped at 5MB with strings only. **IndexedDB** is an asynchronous, transactional, indexed NoSQL datastore that handles large binary blobs (like our steganographic canvas images) and complex queries without freezing the page."

> 🚀 **You are 100% prepared!**
> Keep this guide handy during the viva. When answering, speak clearly, reference the architecture terms confidently, and emphasize the **Steganography Novelty**. Good luck!